

Matrix Approximation Methods

Deep-Understanding Study Guide

Section I, Questions 1-8

From-zero foundations, algorithms, and theory

This document expands each of the eight questions from Section I of the exam guide into a self-contained, from-zero treatment. Every section opens with prerequisite ideas (matrix factorisations, subspaces, norms, projections), then assembles those pieces into the method itself, and ends with the theorems and practical estimates that an examiner is most likely to probe.

Contents

1. QR with column pivoting and rank-revealing QR
2. Randomized SVD
3. The Lanczos method
4. Recompression of low-rank matrices in skeleton format
5. The maximum-volume principle
6. The maxvol algorithm
7. Cross methods
8. Estimating the approximation and the error norm in cross methods

1. QR with column pivoting and rank-revealing QR

1.1 Foundations: orthogonality and the QR factorisation

An *orthonormal* set of vectors $\{q_1, \dots, q_k\}$ satisfies $q_i^T q_j = 1$ if $i = j$ and 0 otherwise. Stacked as columns of a matrix Q , this means $Q^T Q = I$. Orthonormal bases are central in numerical linear algebra because multiplication by an orthogonal matrix preserves the Euclidean (2-) norm and the Frobenius norm, and therefore does not amplify rounding errors.

The thin QR factorisation of an $m \times n$ matrix A with $m \geq n$ writes

$$A = Q R$$

where $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns and $R \in \mathbb{R}^{n \times n}$ is upper triangular. Geometrically, the k -th column of Q is an orthonormal direction whose span equals the span of the first k columns of A ; the entries of R record how the columns of A are written in this orthonormal basis.

QR is computed by *Householder reflections*, *Givens rotations*, or *modified Gram-Schmidt*. Householder is the workhorse: each reflection zeroes the entries below the diagonal in one column at a time, and the cumulative product of reflections is Q . The cost is roughly $2mn^2 - (2/3)n^3$ floating-point operations.

1.2 Why plain QR is not enough for rank questions

Suppose A has numerical rank $r < n$: that is, r singular values are large and the rest are tiny. Plain QR produces a triangular R whose diagonal entries do not, in general, decrease in magnitude. The information about the rank gap is hidden in R , mixed across both rows and columns. We need the algorithm to *expose* the gap: ideally the leading $r \times r$ block of R is well-conditioned and the trailing block is small.

1.3 QR with column pivoting

QR with column pivoting introduces a permutation matrix P that rearranges columns greedily. The factorisation becomes

$$A P = Q R$$

At step k the algorithm computes the 2-norms of the remaining (unreduced) parts of all columns and swaps the column with the **largest** remaining norm into position k . Then it applies a Householder reflection to zero out everything below the diagonal in that position. This greedy choice ensures that the diagonal entries $|R_{11}|, |R_{22}|, \dots, |R_{nn}|$ form a non-increasing sequence.

Block form: partition R after the r -th column,

$$A P = Q \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix}$$

with $R_{11} \in \mathbb{R}^{r \times r}$ upper triangular and non-singular, $R_{12} \in \mathbb{R}^{r \times (n-r)}$, and $R_{22} \in \mathbb{R}^{(n-r) \times (n-r)}$. The two key quality measures are: (i) how small $\|R_{22}\|$ is compared with $\sigma_{r+1}(A)$, and (ii) how well-conditioned R_{11} is, i.e. how close its smallest singular value $\sigma_{\min}(R_{11})$

is to $\sigma_r(A)$.

1.4 Why column pivoting is not always rank-revealing

The famous **Kahan matrix** shows that column pivoting can be deceived. The Kahan matrix is bidiagonal, all diagonal entries equal to a common value, off-diagonals scaled by a parameter, and the n -th singular value is exponentially small in n . Column pivoting at every step accepts the natural column ordering (no swap is favourable by the local criterion), and the resulting R_{nn} is much larger than $\sigma_n(A)$. The diagonal of R drops slowly, then stops, while the actual smallest singular value is several orders of magnitude smaller.

The lesson: greedy column-norm pivoting is necessary but not sufficient. We need a globally aware criterion.

1.5 Strong rank-revealing QR (Gu and Eisenstat)

A factorisation $A = P = Q R$ is called **strong rank-revealing of order r** if there exist modest constants $q_1(n, r)$ and $q_2(n, r)$ such that

$$\sigma_{\min}(R_{11}) \geq \sigma_r(A) / q_1$$

$$\sigma_{\max}(R_{22}) \leq \sigma_{r+1}(A) \cdot q_2$$

and every entry of $R_{11}^{-1} R_{12}$ is bounded by a small constant.

Gu and Eisenstat construct such factorisations by post-processing an initial pivoted QR. The algorithm repeatedly checks whether swapping a column from the trailing $(n-r)$ block with one from the leading r block would multiply the volume of the leading block by more than a fixed threshold $f > 1$; if so, the swap is performed. Each swap can be carried out with $O((m+n)(n-r))$ work using the explicit formula for how the determinant of the leading minor changes. The key inequality bounds the entries of $R_{11}^{-1} R_{12}$, which controls how much the leading block can be improved.

1.6 What to remember for the exam

- Be able to write $A = P = Q R$ and the 2×2 block form with R_{11} , R_{12} , R_{22} .
- Greedy column-norm pivoting yields a non-increasing diagonal in R and is cheap, but the Kahan matrix shows it can fail to expose the rank.
- Strong RRQR (Gu-Eisenstat) post-processes by swapping columns across the rank boundary so long as a determinant ratio exceeds a threshold $f > 1$; this gives polynomial bounds on $\sigma_{\min}(R_{11})$ and $\sigma_{\max}(R_{22})$ in terms of the true singular values.
- A small $\|R_{22}\|$ together with a well-conditioned R_{11} certifies that the column space of the selected columns is a good approximation of the dominant range of A .

2. Randomized SVD

2.1 Foundations: SVD and the best low-rank approximation

Every matrix $A \in \mathbb{R}^{m \times n}$ has a singular value decomposition

$$A = U \Sigma V^T = \sum \sigma_i u_i v_i^T$$

with orthogonal U , V and a diagonal Σ of singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. The **Eckart-Young theorem** states that the best rank- k approximation of A in the Frobenius (or spectral) norm is the truncated SVD

$$A_k = \sum_{i=1..k} \sigma_i u_i v_i^T$$

with error $\|A - A_k\|_F^2 = \sum_{j>k} \sigma_j^2$ and $\|A - A_k\|_2 = \sigma_{k+1}$. The full SVD costs $O(mn \cdot \min(m,n))$ operations, which is prohibitive for very large dense matrices.

2.2 The randomized idea

If we knew, even approximately, an orthonormal basis $Q \in \mathbb{R}^{m \times k}$ for the dominant range of A , then $A \approx Q Q^T A$ would already be a good rank- k approximation. Computing the SVD of the small $k \times n$ matrix $B = Q^T A$ is cheap. So the question is: how do we discover such a basis without doing a full SVD?

Halko, Martinsson and Tropp's answer is to *sketch* the range with a random Gaussian test matrix $\Omega \in \mathbb{R}^{n \times l}$ with $l = k + p$:

$$Y = A \Omega, \text{ then QR: } Y = Q R$$

Because the columns of Ω are random, with overwhelming probability they have non-zero components along all the dominant right singular vectors of A ; multiplying by A amplifies these components by $\sigma_1, \dots, \sigma_k$ and damps the rest. The column space of Y therefore captures the dominant range of A , and an economy QR extracts an orthonormal basis Q .

2.3 The basic algorithm

- **Step 1.** Draw $\Omega \in \mathbb{R}^{n \times l}$ with i.i.d. standard normal entries; set $l = k + p$.
- **Step 2.** Compute the sketch $Y = A \Omega$.
- **Step 3.** Orthogonalise: $Y = Q R$, keep Q .
- **Step 4.** Form the small matrix $B = Q^T A \in \mathbb{R}^{l \times n}$.
- **Step 5.** Compute SVD of B : $B = \tilde{U} \Sigma V^T$.
- **Step 6.** Output $U = Q \tilde{U}$, Σ , V^T ; truncate to rank k .

Cost: two matrix-matrix products with A (steps 2 and 4) at $O(mnl)$ each, plus $O((m+n)l^2)$ for the small SVD and orthogonalisation. When $l \ll \min(m,n)$ this is dramatically cheaper than a full SVD.

2.4 Where the error comes from

The total error decomposes as

$$\|A - Q Q^T A\| = \|(I - P_Q)A\|$$

where $P_Q = Q Q^T$ is the orthogonal projector onto $\text{range}(Q)$. Halko–Martinsson–Tropp prove the expectation bound, in spectral norm,

$$\mathbb{E} \|(I - P_Q)A\|_2 \leq (1 + 4 \sqrt{((k+p)/(p-1)) \cdot \sqrt{\min(m,n)}}) \cdot \sigma_{k+1}$$

Already with a small oversampling parameter $p = 5$ or 10 , the constant becomes a moderate multiple of $\sqrt{\min(m,n)}$. The Frobenius-norm bound is even tighter: with high probability the Frobenius error is within a factor of order $\sqrt{((k+p)/p)}$ of the optimal Eckart–Young error.

2.5 Power iteration: needed when singular values decay slowly

If σ_{k+1}/σ_k is close to 1, the unwanted singular components do not get pushed down enough by a single application of A . The remedy is to use the matrix

$$A_q = (A A^T)^q A$$

which has the same singular vectors as A but singular values σ_j^{2q+1} . The ratio $(\sigma_{k+1}/\sigma_k)^{2q+1}$ can be made tiny by a small q (typically 1 or 2). We replace step 2 with

$$Y = A (A^T A)^q \Omega,$$

with intermediate orthogonalisations after each application of A or A^T to avoid loss of accuracy in floating-point arithmetic. This is called the *randomized subspace iteration*.

2.6 What to remember for the exam

- Sketching with a Gaussian Ω captures the dominant range of A .
- Three pieces: random sketch $Y = A\Omega$, range basis Q from $\text{QR}(Y)$, small SVD of $B = Q^T A$.
- Oversampling $p \approx 5$ – 10 buys robustness; the expectation bound on the spectral-norm error is roughly $(1 + \sqrt{(m \text{ or } n)}) \sigma_{k+1}$.
- Power iteration $q = 1, 2$ corrects slow singular-value decay by raising the gap to the $(2q+1)$ -th power.

3. The Lanczos method

3.1 Foundations: Krylov subspaces

Given a square matrix A and a starting vector $b \neq 0$, the *Krylov subspace* of dimension m is

$$\mathbf{K}_m(A, b) = \text{span}\{ b, Ab, A^2b, \dots, A^{m-1}b \}$$

Krylov spaces are powerful for two reasons. First, they are nested: $\mathbf{K}_m \subset \mathbf{K}_{m+1}$. Second, their basis can be built iteratively using only matrix-vector products with A ; we never need to factorise A or even store it densely. This is the key to large-scale eigenvalue and singular value computation.

The **Arnoldi process** applies modified Gram-Schmidt to the vectors $b, Ab, A^2b, \dots$ producing an orthonormal basis $q_1, q_2, \dots$ and an upper Hessenberg matrix H_m with

$$A Q_m = Q_m H_m + h_{m+1,m} q_{m+1} e_m^T$$

3.2 Symmetric Lanczos

When A is symmetric ($A = A^T$), the Hessenberg matrix $H_m = Q_m^T A Q_m$ is itself symmetric, and a symmetric Hessenberg matrix is tridiagonal:

$$T_m = \text{tridiag}(\beta_{i-1}, \alpha_i, \beta_i)$$

Substituting this into the Arnoldi recurrence gives a beautiful **three-term recurrence**:

$$A q_k = \beta_{k-1} q_{k-1} + \alpha_k q_k + \beta_k q_{k+1}$$

with $\alpha_k = q_k^T A q_k$ and β_k chosen to renormalise. Storage is just three vectors. After m steps, we have

$$A Q_m = Q_m T_m + \beta_m q_{m+1} e_m^T$$

Eigenvalues of the small tridiagonal T_m , called **Ritz values**, are excellent approximations to the extreme eigenvalues of A ; their convergence is governed by Chebyshev polynomials and is rapid when the wanted eigenvalues are well-separated from the rest.

3.3 Loss of orthogonality

In exact arithmetic the Lanczos vectors are mutually orthogonal. In floating-point arithmetic, however, as soon as a Ritz value converges, the Lanczos vectors begin to lose orthogonality and previously converged eigenvalues reappear as **spurious copies** (the so-called ‘ghost eigenvalues’). Three remedies are common:

- **Full reorthogonalisation**: at each step subtract the components of the new vector along all previous Lanczos vectors. Robust but expensive ($O(mk)$ memory).
- **Selective reorthogonalisation** (Parlett-Scott): reorthogonalise only against converged Ritz vectors, detected by monitoring the residuals.
- **Implicit restarts** (IRLM, ARPACK): periodically project the process onto a smaller, refreshed Krylov space to keep memory bounded.

3.4 Golub-Kahan bidiagonalisation (Lanczos for SVD)

For singular values, Lanczos is applied to either $A^T A$ or to the Jordan-Wielandt block $\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}$, or directly via **Golub-Kahan bidiagonalisation**: starting from a unit vector u_1 , generate U_k and V_k such that

$$A V_k = U_{k+1} B_k, A^T U_k = V_k B_k^T$$

where B_k is lower bidiagonal. The singular values of B_k are the **Ritz singular values** of A , and a small SVD of B_k (cost $O(k)$ per iteration of dqds) yields the wanted approximations.

3.5 What to remember for the exam

- Krylov subspace $\mathbf{K}_m(A, b)$ and the Arnoldi recurrence $A Q_m = Q_m H_m + \text{residual}$.
- Symmetry collapses Hessenberg to tridiagonal: three-term recurrence, only three vectors stored at a time.
- Loss of orthogonality is fundamental in finite precision; remedy by full / selective reorthogonalisation or restarts.
- Golub-Kahan bidiagonalisation is the singular-value version: the small bidiagonal B_k gives Ritz singular values that converge to those of A from outside in.

4. Recompression of low-rank matrices in skeleton format

4.1 Foundations: skeleton (CUR) representation

Suppose a matrix $A \in \mathbb{R}^{m \times n}$ has rank exactly r . Pick any r linearly independent rows (with row indices I) and any r linearly independent columns (with column indices J). The **intersection matrix** is

$$A(I, J) \in \mathbb{R}^{r \times r}, \text{ nonsingular.}$$

Then the exact **skeleton (or CUR) decomposition** holds:

$$A = C U R, C = A(:, J), U = A(I, J)^{-1}, R = A(I, :)$$

Storage is $r(m+n-r)$ entries, identical to a rank- r SVD form. Computation needs only r rows and r columns of A , which is invaluable when A is given by a procedure that computes one entry at a time.

For rank- r approximation rather than exact representation, Goreinov-Tyrtysnikov-Zamarashkin's pseudo-skeleton theorem guarantees that a wisely chosen submatrix $A(I, J)$ yields

$$\|A - C U R\|_C \leq (1 + r)^2 \cdot \sigma_{r+1}(A)$$

in the Chebyshev (entrywise max) norm, when $A(I, J)$ has maximum volume among $r \times r$ submatrices (more on this in questions 5 and 6).

4.2 Why recompression is needed

Skeleton format is closed under no algebraic operation in a friendly way:

- Addition of two rank- r skeletons gives a rank- $(2r)$ skeleton, even if the true rank of the sum is much smaller.
- Hadamard (entrywise) product can produce up to rank $r_1 r_2$.
- Many cross-iteration steps re-add or re-stack pieces, doubling the rank each time.

Without recompression, ranks grow rapidly and the format loses its memory advantage. Recompression rebuilds a more compact representation of the same matrix to a desired tolerance.

4.3 Recompression via thin SVDs

A clean recipe: starting from $A \approx C U R$ with $C \in \mathbb{R}^{m \times r}$, $R \in \mathbb{R}^{r \times n}$, and an $r \times r$ middle U :

- Compute thin QR factorisations: $C = Q_C R_C$, $R^T = Q_R R_R$.
- Form the small middle matrix $M = R_C^T U R_R^T \in \mathbb{R}^{r \times r}$.
- SVD of M : $M = \tilde{U} \Sigma \tilde{V}^T$; truncate to keep singular values above the tolerance ε to obtain rank $\tilde{r} \leq r$.
- Set new factors: $C = Q_C \tilde{U}_{:,1:\tilde{r}}$, $R = \Sigma_{1:\tilde{r}} \tilde{V}_{:,1:\tilde{r}}^T Q_R^T$.

Total cost is $O((m+n)r^2 + r^3)$, which is far cheaper than recomputing an SVD of the full matrix and exactly Eckart-Young optimal in the Frobenius norm.

4.4 Recompression via maxvol

An alternative, often used inside cross algorithms, is to keep the $C U R$ structure and only swap the indices I, J to a smaller maximum-volume submatrix. The maxvol algorithm (question 6) finds new $\tilde{r}$ -row and $\tilde{r}$ -column index sets such that the bound from Goreinov-Tyrtysnikov still applies. This is preferred when computing entries of A is more convenient than manipulating dense SVDs of the small matrix.

4.5 What to remember for the exam

- Skeleton (CUR) form: $A \approx C U R$, with C selected columns, R selected rows, U the inverse of the intersection submatrix; the intersection must be non-singular.
- Why recompress: arithmetic operations grow ranks rapidly.
- SVD-based recompression with the QR-then-SVD-on-the-middle trick costs $O((m+n)r^2)$ and is optimal in Frobenius norm.
- Maxvol-based recompression keeps the CUR structure and uses the $(1+r)^2$ error bound.

5. The maximum-volume principle

5.1 Foundations: volume of a square matrix

The **volume** of a square matrix $X \in \mathbb{R}^{r \times r}$ is simply

$$\text{vol}(X) = |\det X|$$

Geometrically, $|\det X|$ equals the r -dimensional volume of the parallelepiped spanned by the rows (equivalently, the columns) of X . Volume is also the product of the singular values:

$$\text{vol}(X) = \sigma_1(X) \cdot \sigma_2(X) \cdot \dots \cdot \sigma_r(X)$$

So a large volume guarantees that the smallest singular value cannot be too small, i.e. the matrix is well-conditioned. This is the intuition behind the principle.

5.2 The principle, stated

Among all $r \times r$ submatrices $A(I, J)$ of A , choose one with **maximum volume**:

$$I^*, J^* = \operatorname{argmax}_{|I|=|J|=r} |\det A(I, J)|$$

Use the corresponding columns and rows in the skeleton formula. The **maximum-volume principle** asserts that this choice nearly minimises the skeleton approximation error.

5.3 Goreinov-Tyrtshnikov bound

For the skeleton built from a maximum-volume $r \times r$ submatrix:

$$\|A - A(:, J) A(I, J)^{-1} A(I, :)\|_C \leq (1 + r) \sigma_{r+1}(A)$$

in the entrywise Chebyshev norm, where $\sigma_{r+1}(A)$ is the $(r+1)$ -th singular value, i.e. the best achievable rank- r error in spectral norm. The proof uses Cramer's rule and the fact that interpolation coefficients $A(:, J) A(I, J)^{-1}$ have entries bounded in absolute value by 1 when $|\det A(I, J)|$ is maximum.

Why the bound has this form. Set $F = A(:, J) A(I, J)^{-1}$: this is an $m \times r$ matrix of *interpolation coefficients*. By the max-volume property, every entry of F satisfies $|F_{ij}| \leq 1$ (otherwise swapping rows would increase the volume). Now write the residual at any fixed entry (i, j) using a low-rank perturbation argument; the residual is bounded by $\sigma_{r+1}(A)$ times the $(r+1)$ -st minor of an enlarged $(r+1) \times (r+1)$ matrix, which contributes the factor $(1 + r)$.

5.4 Why determinants couple to conditioning

If a submatrix has very small volume, then its smallest singular value is tiny (since the volume is the product of all singular values), the inverse $A(I, J)^{-1}$ has huge norm, and the interpolation amplifies any error in the entries. Conversely the maximum volume guarantees that the inverse cannot blow up. This is the linear-algebra heart of skeleton/cross approximation.

5.5 What to remember for the exam

- Volume = $|\det|$ = product of singular values; large volume forces the smallest singular value to be $\geq \text{vol} / (\text{largest singular value})^{r-1}$.
- The skeleton formed from a maximum-volume $r \times r$ submatrix satisfies the entrywise bound $(1 + r) \sigma_{r+1}$.
- Interpolation coefficients $A(:, J) A(I, J)^{-1}$ have entries of magnitude ≤ 1 when the chosen submatrix has maximum volume.
- The principle generalises to tensors: maxvol-based nested interpolation underlies TT-cross and skeleton-cross algorithms.

6. The maxvol algorithm

6.1 Why a heuristic is needed

Finding a globally maximum-volume submatrix is NP-hard in general (it is closely related to the maximum-determinant problem on bipartite graphs). Fortunately, for skeleton approximation we do not need the global maximum: the $(1 + r)$ bound only uses the fact that no row or column swap inside a single block can increase the volume. This is a *local* condition, and there is a fast greedy algorithm that achieves it. This is the **maxvol algorithm** of Goreinov, Oseledets, Savostyanov, Tyrtyshnikov and Zamarashkin (2010).

6.2 The setting

We are given a tall matrix $B \in \mathbb{R}^{n \times r}$ with $n \gg r$ (think of B as the columns $A(:, J)$). The goal is to choose r rows of B indexed by $I \subset \{1, \dots, n\}$ so that $B(I, :)$ has volume locally maximal.

Local maximality is conveniently expressed through the **coefficient matrix**

$$C = B \cdot B(I, :)^{-1} \in \mathbb{R}^{n \times r}$$

The rows of C indexed by I are the rows of the $r \times r$ identity. For any other row $i \notin I$, the entry $C_{i, k}$ tells us how the volume changes if row i replaces row I_k :

$$\text{vol after swap} / \text{vol before swap} = |C_{i, k}|$$

Therefore the volume can be increased *iff* some entry of C exceeds 1 in absolute value.

6.3 The algorithm

- **Step 0.** Initialise I from any $r \times r$ non-singular submatrix (e.g. the first r rows after column pivoting in a QR factorisation of B).
- **Step 1.** Compute $C = B \cdot B(I, :)^{-1}$.
- **Step 2.** Find the entry of largest absolute value, $(i^*, k^*) = \operatorname{argmax}_{i, k} |C_{i, k}|$.
- **Step 3.** If $|C_{i^*, k^*}| \leq 1 + \delta$ for a tolerance $\delta > 0$, stop.
- **Step 4.** Otherwise replace I_{k^*} by i^* , update C by a rank-one correction (no recomputation of the inverse), and return to step 2.

The rank-one update of C uses the Sherman-Morrison formula and costs $O(nr)$ per step. The algorithm is guaranteed to terminate because the volume strictly increases at each step and is bounded above by the maximum over all $C(n, r)$ submatrices.

6.4 Quality of the local maximum

Let v_{local} be the volume found by maxvol and v_{max} the global maximum. The local termination condition guarantees

$$v_{\text{local}} \geq (1 + \delta)^{-r} v_{\text{max}}$$

and, more importantly, the entries of the coefficient matrix C are all bounded by $1 + \delta$, so the skeleton bound becomes

$$\|A - C U R\|_C \leq (1 + \delta)(1 + r) \sigma_{r+1}(A)$$

Using a small δ (e.g. $\delta = 0.05$) gives essentially the same theoretical guarantee as the true maximum volume, at modest practical cost.

6.5 Where maxvol is used

- Inside **cross approximation** (question 7) for choosing informative rows and columns.
- Inside **TT-cross** for nested index sets along a tensor train; each sweep uses maxvol on a reshape of three or four cores.
- As a **recompression tool** for skeleton-format matrices (question 4).
- For **strong RRQR**, where the swap criterion is essentially the same coefficient-matrix condition.

6.6 What to remember for the exam

- Maxvol is a *local* greedy algorithm: stop when $\|C\|_{\max} \leq 1 + \delta$.
- The global maximum-volume problem is hard, but the local condition is what the $(1 + r)$ bound actually requires.
- Each iteration is one row swap with $O(n r)$ work via Sherman-Morrison; convergence in a small number of swaps in practice.
- The output supports a clean error bound $(1 + \delta)(1 + r) \sigma_{r+1}$ for the skeleton.

7. Cross methods

7.1 The setting that motivates everything

Suppose we are given an enormous matrix $A \in \mathbb{R}^m \times n$ that we cannot afford to assemble entirely (perhaps a kernel matrix from a boundary-element method for a 3D scattering problem, with m, n in the millions), but we can compute any single entry $A(i, j)$ on demand at moderate cost. We also know or strongly suspect that A is well approximated by a low-rank matrix. We want to construct a rank- r approximation by reading only a small number of entries — say $O((m+n)r)$.

SVD is unavailable: it requires every entry. Rank-revealing QR is unavailable: it requires every column. Cross methods are designed exactly for this regime.

7.2 The cross (skeleton) approximation

Pick row indices $I \subset \{1, \dots, m\}$ and column indices $J \subset \{1, \dots, n\}$, both of size r , such that the intersection submatrix $A(I, J)$ is non-singular. Form

$$A \approx A(:, J) \cdot A(I, J)^{-1} \cdot A(I, :)$$

This formula is exact when A has rank $\leq r$ and the intersection is non-singular. For approximately low-rank A , by question 5 the error is at most $(1 + r) \cdot \sigma_{r+1}(A)$ in the Chebyshev norm *provided* the intersection has near maximum volume. Computing the approximation needs only $m r + r n - r^2$ entries of A , plus an $r \times r$ matrix inversion.

7.3 Adaptive cross approximation (Bebendorf)

How do we find a good I, J without computing the full A ? Bebendorf's **adaptive cross approximation (ACA)** builds the approximation incrementally, using residual pivoting. Initialize $R_0 = A$ (only conceptually, never stored). For $k = 1, 2, \dots$:

- Choose a pivot row i_k (for instance, the row with the largest residual seen so far, or sequentially).
- Compute the row $r_k = R_{k-1}(i_k, :)$ using only the previously stored vectors and one full row of A .
- Find the column index j_k with the largest magnitude entry in r_k : $j_k = \operatorname{argmax}_j |r_k(j)|$.
- Compute the column $c_k = R_{k-1}(:, j_k)$ and rescale to obtain $u_k = c_k / r_k(j_k)$ and $v_k = r_k$.
- Update $R_k = R_{k-1} - u_k v_k^T$; again only conceptually, but its entries are computed on demand as $A(i, j) - \sum_{1 \leq l \leq k} u_l(i) v_l(j)$.

After r steps the approximation is $A_r = \sum_{k=1..r} u_k v_k^T$, a rank- r matrix in outer-product form. The pivot rows $I = \{i_1, \dots, i_r\}$ and columns $J = \{j_1, \dots, j_r\}$ constitute the chosen cross. Each step touches one row and one column of A , costing $O(m+n)$ memory and the same number of entry queries.

7.4 Tyrtysnikov's mosaic-skeleton method

When the matrix is not globally low-rank but has block low-rank structure (typical for kernel matrices from non-local operators), Tyrtyshnikov's **mosaic-skeleton method** partitions the matrix into a hierarchy of admissible blocks (well-separated source/target clusters) and applies cross approximation to each block separately. The data-sparse output is the engine behind hierarchical matrices ($\mathcal{H}$ -matrices) and panel-clustering techniques.

7.5 Connection to the global maxvol view

ACA's pivoting rule corresponds to local greedy maximisation of a *partial* volume after each step. After all r steps, the resulting cross may not be globally maxvol, but in practice, and in theory under mild assumptions (Bebendorf, Tyrtyshnikov 2000), the error decays at almost the same rate as the optimal SVD.

7.6 Differences from SVD

- SVD reads the entire matrix; cross reads $O((m + n) r)$ entries.
- SVD gives the optimal rank- r approximation in 2-norm; cross is quasi-optimal up to a $(1 + r)$ factor (Chebyshev norm) or similar Frobenius-norm bounds.
- SVD output is dense in singular vectors; cross output uses original rows and columns of A , which may be more interpretable.
- Cross is much cheaper, especially for matrices defined by formulae or function evaluations.

7.7 What to remember for the exam

- Cross / skeleton formula $A \approx A(:, J) A(I, J)^{-1} A(I, :)$, exact when $\text{rank}(A) \leq r$ and the intersection is non-singular.
- Adaptive cross approximation (ACA) builds the rank- r approximation greedily, using $O((m+n)r)$ queries.
- Mosaic-skeleton extends the idea to block-low-rank kernel matrices.
- Cross is the matrix-level counterpart of TT-cross for tensors.

8. Estimating the approximation norm and the error norm in cross methods

8.1 The fundamental difficulty

In a standard SVD-based truncation we have direct access to the singular values, so the discarded tail is known exactly: $\|A - A_r\|_F^2 = \sum_{j>r} \sigma_j^2$. In cross methods we never form the residual matrix; that would require all $m \cdot n$ entries of A . Therefore the error must be **estimated a posteriori** from a small subset of entries, balancing reliability against cost.

8.2 The norm of the approximation

Computing $\|A_r\|_F$ is by contrast straightforward, because we have the cross factors. If $A_r = \sum_{k=1..r} u_k v_k^T$ then

$$\|A_r\|_F^2 = \sum_{i,j} \langle u_i, u_j \rangle \langle v_i, v_j \rangle$$

computed as a sum of r^2 dot products, total cost $O((m + n)r^2)$. Equivalently, if $U = [u_1 \dots u_r]$ and $V = [v_1 \dots v_r]$, the squared Frobenius norm equals the trace of the Gram product $(U^T U) (V^T V)$. This is the foundation of relative-tolerance criteria: we monitor $\|u_k\| \|v_k\|$ relative to $\|A_r\|$ to decide when to stop.

8.3 The norm of the error: a posteriori estimators

We list the four families of estimators commonly used inside ACA and TT-cross.

Estimator 1: residual probing on test entries

Pick a small random set of test indices $T = \{(i_1, j_1), \dots, (i_N, j_N)\}$ with $|T| \ll m \cdot n$. Compute the entries of A and of A_r at these positions and form the empirical estimate

$$\|A - A_r\|_F^2 \approx (m \cdot n / |T|) \cdot \sum_{(i,j) \in T} (A_{ij} - A_{r,ij})^2$$

By Hoeffding/Chebyshev concentration, $|T| \approx 100\text{--}1000$ already gives a reliable order-of-magnitude estimate. The cost is just $|T|$ extra entry queries.

Estimator 2: stabilization of the chosen cross fibers

In ACA, the magnitudes of the new pivot vectors u_k, v_k shrink as the algorithm captures the dominant content of A . If the geometric mean of the last few outer-product norms $\|u_k\| \|v_k\|$ falls below $\varepsilon \cdot \|A_r\|_F$, the iteration is stopped. This is essentially a relative-tolerance criterion driven by the Cauchy-Schwarz inequality:

$$\|A - A_r\|_F \leq \sum_{k>r} \|u_k\| \|v_k\| \text{ (heuristic surrogate)}$$

Estimator 3: maxvol-based bound

Once the row and column index sets I, J have been selected to satisfy the local maxvol condition (entries of $A(:, J) A(I, J)^{-1}$ bounded by $1 + \delta$), Goreinov-Tyrtshnikov's bound gives

$$\|A - A_r\|_C \leq (1 + \delta)(1 + r) \sigma_{r+1}(A)$$

This is rigorous, but it requires σ_{r+1} , which is unknown. In practice σ_{r+1} is itself estimated, e.g. by the smallest singular value found in a small randomized SVD on a test subblock.

Estimator 4: norm of the interpolation coefficient matrix

The matrix $F = A(:, J) A(I, J)^{-1}$ of interpolation coefficients amplifies any noise in the chosen rows; its 2-norm is a useful surrogate for the conditioning of the skeleton:

$$\text{error} \leq \|F\|_2 \cdot \|A - \Pi_r A\|$$

with Π_r the orthogonal projector onto $\text{range}(F)$. Computing $\|F\|_2$ uses only an $r \times r$ SVD after orthogonalising F , so it is essentially free.

8.4 Practical stopping criterion in ACA

The combination most often used in ACA implementations:

- Maintain a running Frobenius norm $\|A_r\|_F$ updated by the Gram-matrix formula above each iteration (cost $O(r(m+n))$).
- Maintain the size $\|u_k\| \|v_k\|$ of the last update.
- Stop when several (say 3 in a row) updates produce a relative size below a user-set tolerance ε .
- Optionally, validate with a random test set of N entries (cost N entry queries).

8.5 Why error depends on submatrix quality

Even if the algorithm has selected exactly the right number of rows r , a poor choice of I , J can blow up the interpolation norm $\|F\|$, and therefore the error. The $(1+r)$ bound only applies when the chosen submatrix is locally maxvol; without that property, the error can be arbitrarily larger than σ_{r+1} . This is the principal reason the maxvol algorithm is so important inside cross methods.

8.6 What to remember for the exam

- Distinguish $\|A_r\|$ (computable cheaply from the factors) from $\|A - A_r\|$ (must be estimated).
- Four families of estimators: random test entries (Monte-Carlo), stabilisation of the latest cross fibers, maxvol-based bound $(1+\delta)(1+r)\sigma_{r+1}$, and the norm of the interpolation matrix F .
- Practical stopping criterion combines a Gram-matrix tracker for $\|A_r\|_F$ with a relative-tolerance test on the latest update size.
- Error quality is dominated by submatrix quality; without maxvol the bound $(1+r)\sigma_{r+1}$ does not hold.

Summary of key formulas

Method	Key formula or estimate
QR pivoting	$A P = Q R$; R block $[R_{11} \ R_{12}; 0 \ R_{22}]$
Strong RRQR	$\sigma_{\min}(R_{11}) \geq \sigma_r(A)/q_1$, $\sigma_{\max}(R_{22}) \leq \sigma_{r+1}(A) q_2$
Randomized SVD	$Y = A\Omega$; $Y = QR$; $B = Q^T A$; $\text{SVD}(B)$
Randomized SVD error	$E\ A - QQ^T A\ _2 \leq (1 + 4\sqrt{((k+p)/(p-1))\sqrt{(\min))}} \sigma_{k+1}$
Lanczos recurrence	$A q_k = \beta_{k-1} q_{k-1} + \alpha_k q_k + \beta_k q_{k+1}$
Skeleton (CUR)	$A \approx A(:, J) A(I, J)^{-1} A(I, :)$
Maxvol bound	$\ A - \text{skeleton}\ _C \leq (1 + r) \sigma_{r+1}(A)$
Maxvol stopping	$\max_{i, k} C_{i, k} \leq 1 + \delta$
ACA update	$u_k = (\text{residual column})/(\text{pivot value})$; $v_k = \text{residual row}$
Frobenius of CUR	$\ A_r\ _F^2 = \text{trace}((U^T U)(V^T V))$

Recommended reading order, in escalation

Foundations first: Trefethen and Bau, *Numerical Linear Algebra*, lectures on QR (pp. 50-83), SVD (pp. 25-45), Krylov / Arnoldi / Lanczos (pp. 235-267).

Specialised theory: Halko, Martinsson and Tropp (2011) for randomized SVD; Gu and Eisenstat (1996) for strong RRQR; Goreinov-Tyrtysnikov-Zamarashkin (1997) for the maxvol principle; Goreinov-Oseledets-Savostyanov-Tyrtysnikov-Zamarashkin (2010) for the maxvol algorithm; Bebendorf (2000) for ACA; Tyrtysnikov (2000) for mosaic-skeleton; Savostyanov (2014) for the quasi-optimal tensor-cross theory.

Reference handbook: Golub and Van Loan, *Matrix Computations* (4th ed.), for QR with column pivoting (pp. 248-288) and Lanczos bidiagonalisation for SVD (pp. 486-507).